

Bloc 2 MVC PHP TD3

Contexte	2
Exercice 1	2
Exercice 2	3
Exercice 3	4
Exercice 4	6
Exercice 5	7
Exercice 6	8
Exercice 7	12
Exercice 8	18
Exercice 9	20
Conclusion	22

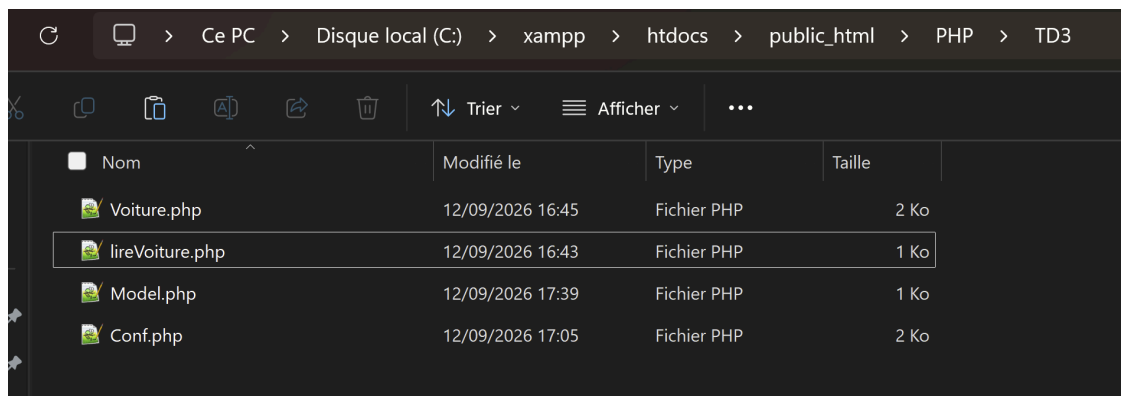
Contexte

Ce TD est la suite du TD2 sur le MVC PHP.

*(Les fichiers projet seront disponibles dans le portfolio sur **GitHub**).*

Exercice 1

Création du dossier “/TD3” qui devra inclure une copie de “**Conf.php**”, “**Model.php**”, “**lireVoiture.php**” et “**Voiture.php**”.



Ajout de cette fonction dans “**Voiture.php**” qui permet d'éviter une injection SQL :

```
public static function getVoitureByImmat($immat) {
    $sql = "SELECT * from voiture WHERE immatriculation=:nom_tag";
    // Préparation de la requête
    $req_prep = Model::$pdo->prepare($sql);

    $values = array(
        "nom_tag" => $immat,
        //nomdutag => valeur, ...
    );

    // On donne les valeurs et on exécute la requête
    $req_prep->execute($values);

    // On récupère les résultats comme précédemment
    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
    $tab_voit = $req_prep->fetchAll();
    // Attention, si il n'y a pas de résultats, on renvoie false
    if (empty($tab_voit))
        return false;
    return $tab_voit[0];
}
```

On va ensuite essayer cette fonction dans “lireVoiture.php” :

```
<?php
require_once 'Model.php';
require_once 'Voiture.php';

$voiture = Voiture::getVoitureByImmat('AA-000-AA');

echo "<h1>Recherche de voiture</h1>";

if ($voiture === false) {
    echo "<p>Aucune voiture trouvée avec cette immatriculation.</p>";
} else {
    $voiture->afficher();
}
?>
```

Résultat du test :



Exercice 2

Ajout de la fonction “**save()**” dans “**Voiture.php**” qui permet d'insérer la voiture courante dans la **BDD**.

```
public function save() {
    $sql = "INSERT INTO voiture (marque, couleur, immatriculation) VALUES (:marque, :couleur, :immatriculation)";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "marque" => $this->marque,
        "couleur" => $this->couleur,
        "immatriculation" => $this->immatriculation
    );
    $req_prep->execute($values);
}
```

On va ensuite instancié un objet de la classe et utilisé “**save()**” (dans “lireVoiture.php”) :

```
$nouvelleVoiture = new Voiture("Peugeot", "Rouge", "XX-999-YY");  
$nouvelleVoiture->save();
```

La voiture est bien été ajouté dans la **BDD (phpMyAdmin)** :

				immatriculation	marque	couleur
☐	Éditer	Copier	Supprimer	AA-000-AA	Peugeot	Rouge
☐	Éditer	Copier	Supprimer	XX-999-YY	Peugeot	Rouge

Exercice 3

On récupère “**creerVoiture.php**” et “**formulaireVoiture.html**” du TD1

Nom	Modifié le	Type	Taille
 Conf.php	12/09/2026 17:05	Fichier PHP	2 Ko
 lireVoiture.php	17/09/2026 14:08	Fichier PHP	1 Ko
 Model.php	12/09/2026 17:39	Fichier PHP	1 Ko
 Voiture.php	17/09/2026 14:07	Fichier PHP	3 Ko
☒  creerVoiture.php	19/05/2026 13:47	Fichier PHP	1 Ko
☒  formulaireVoiture.html	19/05/2026 13:44	Fichier HTML	1 Ko

On modifie “**creerVoiture.php**” de sorte qu’elle sauvegarde l’objet **Voiture** reçu.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> testVoiture </title>
  </head>
  <body>
    <?php
    if (isset($_POST['marque']) && isset($_POST['couleur']) && isset($_POST['immatriculation'])) {

      $marque = $_POST['marque'];
      $couleur = $_POST['couleur'];
      $immatriculation = $_POST['immatriculation'];

      echo "<li>" . htmlspecialchars($marque) . "</li>";
      echo "<li>" . htmlspecialchars($couleur) . "</li>";
      echo "<li>" . htmlspecialchars($immatriculation) . "</li>";
      require_once('Voiture.php');

      // New
      require_once('Model.php');
      try {
        $voiture = new Voiture($marque, $couleur, $immatriculation);
        $voiture->save();
      } catch (PDOException $e) {
        echo "Erreur : " . $e->getMessage();
      }
    }

    ?>
  </body>
</html>
```

On teste de remplir le formulaire :

Mon formulaire :

Marque :

Couleur :

Immatriculation :

Lorsqu'on clique sur envoyer et qu'on vérifie la **BDD**, on peut voir que la voiture a bien été ajouté :

				immatriculation	marque	couleur
☐	Éditer	Copier	Supprimer	00-000-00	Mickey	Turquoise
☐	Éditer	Copier	Supprimer	AA-000-AA	Peugeot	Rouge
☐	Éditer	Copier	Supprimer	XX-999-YY	Peugeot	Rouge

Exercice 4

Les tables **trajet** et **utilisateur** ont été créé dans le **TD2** :



J'y est ajouté quelques données.

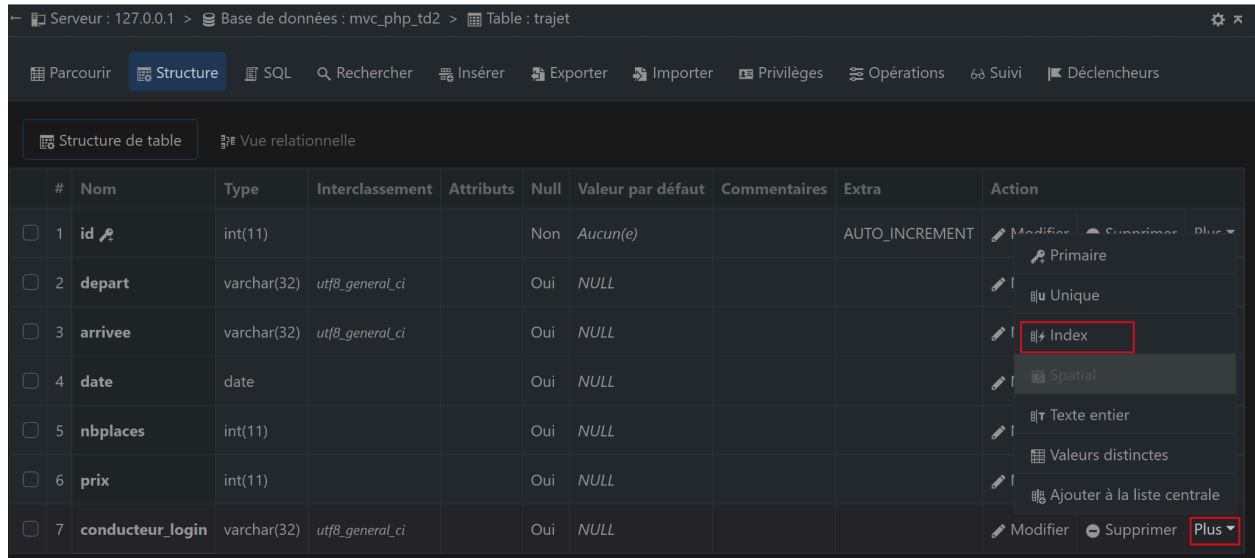
Question : "On souhaite que le champ **trajet.conducteur_login** corresponde à tout moment à un login de conducteur **utilisateur.login**. Vous souvenez-vous quelle est la fonctionnalité des bases de données qui permet ceci ?"

Réponse : C'est la clé étrangère.

Exercice 5

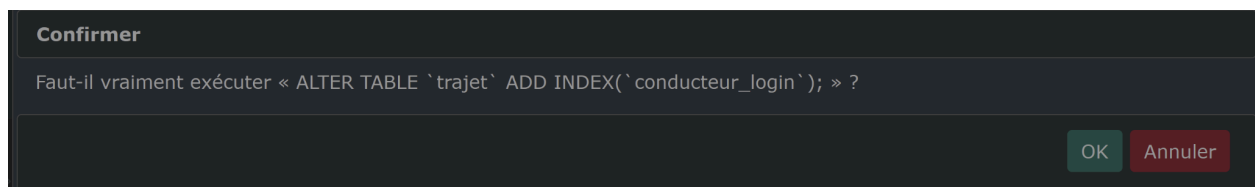
On va faire les liens :

On va dans la table **Trajet** et on va dans "**Structure**" puis on sélectionne "**Index**" dans "**conducteur_login**" :



The screenshot shows a database management interface for a table named 'trajet'. The table structure is displayed with columns: #, Nom, Type, Interclassement, Attributs, Null, Valeur par défaut, Commentaires, Extra, and Action. The 'conducteur_login' column is highlighted, and a context menu is open over it, showing options like 'Modifier', 'Supprimer', 'Plus', 'Index', 'Spatial', 'Valeurs distinctes', and 'Ajouter à la liste centrale'. The 'Index' option is selected and highlighted with a red box.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
1	id	int(11)			Non	Aucun(e)		AUTO_INCREMENT	Modifier, Supprimer, Plus, Primaire
2	depart	varchar(32)	utf8_general_ci		Oui	NULL			Modifier, Unique
3	arrivee	varchar(32)	utf8_general_ci		Oui	NULL			Modifier, Index (selected), Spatial
4	date	date			Oui	NULL			Modifier, Texte entier
5	nbplaces	int(11)			Oui	NULL			Modifier, Valeurs distinctes
6	prix	int(11)			Oui	NULL			Modifier, Ajouter à la liste centrale
7	conducteur_login	varchar(32)	utf8_general_ci		Oui	NULL			Modifier, Supprimer, Plus



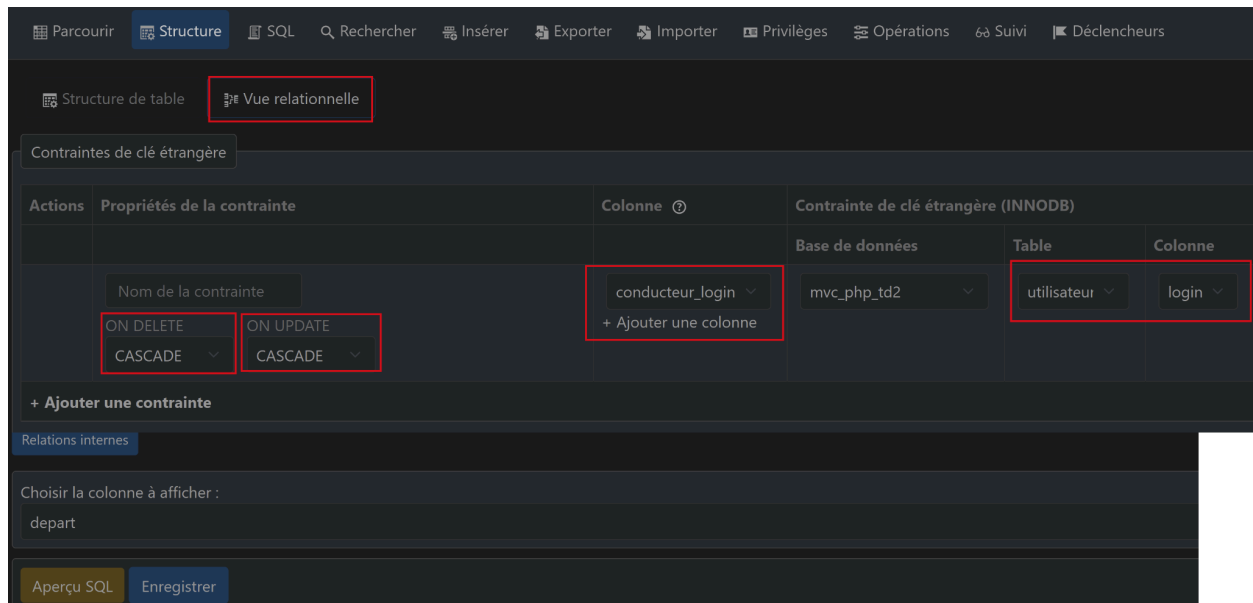
The screenshot shows a confirmation dialog box titled 'Confirmer'. It asks: 'Faut-il vraiment exécuter « ALTER TABLE `trajet` ADD INDEX(`conducteur\_login`); » ?'. There are two buttons: 'OK' and 'Annuler'.

Confirmer

Faut-il vraiment exécuter « ALTER TABLE `trajet` ADD INDEX(`conducteur\_login`); » ?

OK Annuler

Puis dans “**Vue relationnelle**” on sélectionne tous ces paramètres :



Structure de table Vue relationnelle

Contraintes de clé étrangère

Actions	Propriétés de la contrainte	Colonne	Contrainte de clé étrangère (INNODB)		
			Base de données	Table	Colonne
	Nom de la contrainte	conducteur_login	mvc_php_td2	utilisateur	login
	ON DELETE CASCADE	+ Ajouter une colonne			
	ON UPDATE CASCADE				

+ Ajouter une contrainte

Relations internes

Choisir la colonne à afficher :
depart

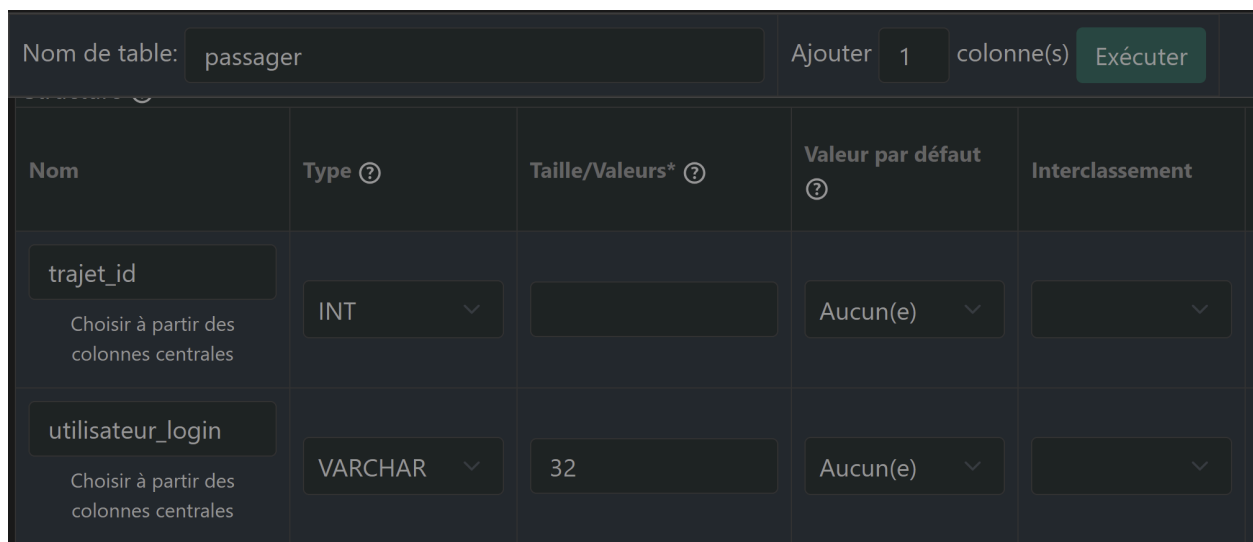
Aperçu SQL Enregistrer

Question : Quelle est la clé primaire de la table passager ?

Réponse : La clé primaire de la table **passager** est une clé primaire composite formée par le couple des deux champs : **trajet_id** et **utilisateur_login**.

Exercice 6

Création de la table **passager** :



Nom de table: passager

Ajouter 1 colonne(s) Exécuter

Nom	Type	Taille/Valeurs*	Valeur par défaut	Interclassement
trajet_id Choisir à partir des colonnes centrales	INT		Aucun(e)	
utilisateur_login Choisir à partir des colonnes centrales	VARCHAR	32	Aucun(e)	

Résultat :

Structure de table											
	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action	
☐	1	trajet_id	int(11)			Non	Aucun(e)			Modifier	Supprimer Plus
☐	2	utilisateur_login	varchar(32)	utf8mb4_general_ci		Non	Aucun(e)			Modifier	Supprimer Plus

On définit les 2 colonnes comme des clés primaires :

[Éditer en ligne] [Éditer] [Créer le code source PHP]											
Structure de table											
	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action	
☒	1	trajet_id	int(11)			Non	Aucun(e)			Modifier	Supprimer Plus
☒	2	utilisateur_login	varchar(32)	utf8mb4_general_ci		Non	Aucun(e)			Modifier	Supprimer Plus
↑ ☒ Tout cocher Avec la sélection : ☐ Parcourir ☐ Modifier ☐ Supprimer ☒ Primaire ☐ Unique ☐ Index ☐ Spatial ☐ Texte entier											

Puis on crée les relations :

Structure de table

Vue relationnelle

Contraintes de clé étrangère

Actions	Propriétés de la contrainte	Colonne	Contrainte de clé étrangère (INNODB)		
			Base de données	Table	Colonne
	Nom de la contrainte ON DELETE ON UPDATE CASCADE CASCADE	trajet_id + Ajouter une colonne	mvc_php_td2	trajet	id
	Nom de la contrainte ON DELETE ON UPDATE CASCADE CASCADE	utilisateur_login + Ajouter une colonne	mvc_php_td2	utilisateur	login
+ Ajouter une contrainte					

On ajoute ensuite des valeurs :

Parcourir Structure SQL Rechercher Insérer Exporter Importer Privilèges Opérations Suivi Déclencheurs

Colonne	Type	Fonction	Null	Valeur
trajet_id	int(11)			Gap - 1
utilisateur_login	varchar(32)			elevex - elevex

Exécuter

☐ Ignorer

Colonne	Type	Fonction	Null	Valeur
trajet_id	int(11)			Gap - 2
utilisateur_login	varchar(32)			jdupont - jdupont

Exécuter

Cela ressemble à cela actuellement :

				trajet_id	utilisateur_login
☐	Éditer	Copier	Supprimer	1	elevex
☐	Éditer	Copier	Supprimer	2	jdupont

On va vérifier si “**CARCADE**” fonctionne, on va créer un jeu de données puis on va le supprimer.

On crée un nouveau trajet avec 2 places

☐	Éditer	Copier	Supprimer	4	Rue des Poitier	Norvège	2026-09-08	2	999	elevex
--------------------------	--------	--------	-----------	---	-----------------	---------	------------	---	-----	--------

et les passagers :

Colonne	Type	Fonction	Null	Valeur
trajet_id	int(11)			Rue des Poitier - 4
utilisateur_login	varchar(32)			elevex - elevex

☐ Ignorer

Colonne	Type	Fonction	Null	Valeur
trajet_id	int(11)			Rue des Poitier - 4
utilisateur_login	varchar(32)			jdupont - jdupont

Exécuter

☐	Éditer	Copier	Supprimer	4	elevex
☐	Éditer	Copier	Supprimer	4	jdupont

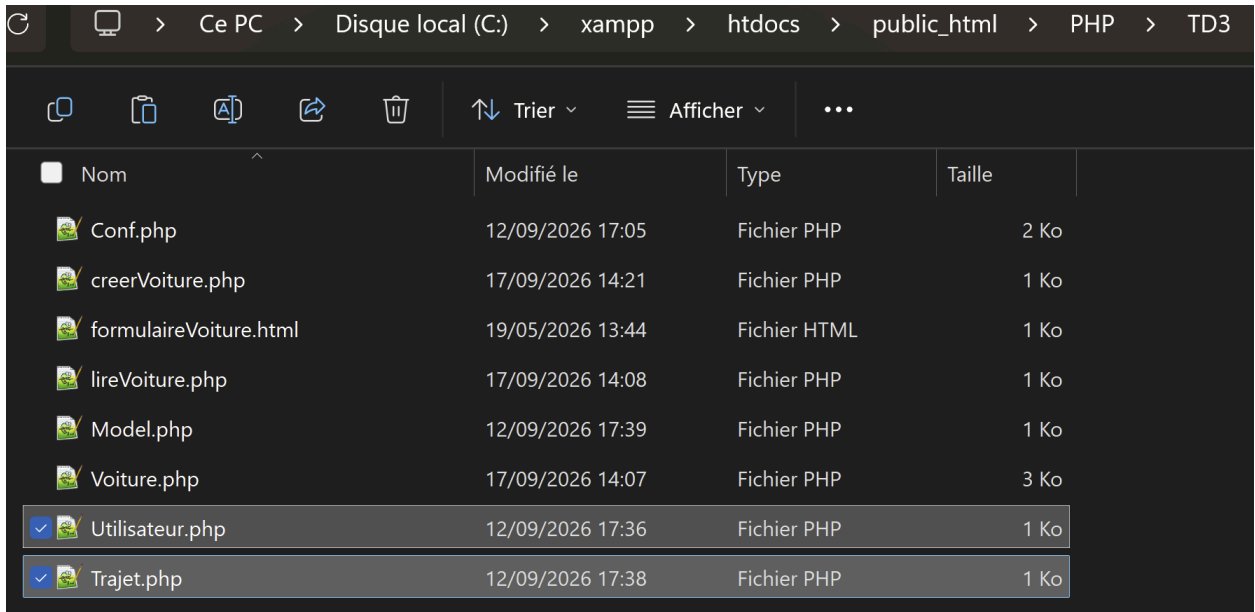
On va supprimer “**elevex**” et vérifie si les 2 lignes au dessus ont été supprimé :

	login	nom	prenom
☐ Éditer Copier Supprimer	elevex	Martin	Lucas
☐ Éditer Copier Supprimer	jdupont	Dupont	Jean
☐ Éditer Copier Supprimer	mbardet	Bardet	Marie

Toutes les lignes où “**elevex**” été conducteur on été supprimé automatiquement de la table **passager**.

Exercice 7

Ajout de “Utilisateur.php” et “Trajet.php” dans “/TD3”



Nom	Modifié le	Type	Taille
Conf.php	12/09/2026 17:05	Fichier PHP	2 Ko
creerVoiture.php	17/09/2026 14:21	Fichier PHP	1 Ko
formulaireVoiture.html	19/05/2026 13:44	Fichier HTML	1 Ko
lireVoiture.php	17/09/2026 14:08	Fichier PHP	1 Ko
Model.php	12/09/2026 17:39	Fichier PHP	1 Ko
Voiture.php	17/09/2026 14:07	Fichier PHP	3 Ko
☒ Utilisateur.php	12/09/2026 17:36	Fichier PHP	1 Ko
☒ Trajet.php	12/09/2026 17:38	Fichier PHP	1 Ko

Ajout de la méthode “afficher()” dans les “Utilisateur.php” :

```
<?php
class Utilisateur {
    private $login;
    private $nom;
    private $prenom;

    public function __construct($l = NULL, $n = NULL, $p = NULL) {
        if (!is_null($l) && !is_null($n) && !is_null($p)) {
            $this->login = $l;
            $this->nom = $n;
            $this->prenom = $p;
        }
    }

    public static function getAllUtilisateurs() {
        $sql = "SELECT * FROM utilisateur";
        $rep = Model::$pdo->query($sql);
        $rep->setFetchMode(PDO::FETCH_CLASS, 'Utilisateur');
        return $rep->fetchAll();
    }

    public function afficher() {
        echo "<p>Passager : " . htmlspecialchars($this->prenom) . " " . $this->nom . " (" . htmlspecialchars($this->login) . ")</p>";
    }
}
```

Ajout d'une nouvelle méthode statique "**findPassagers(\$id)**" avec une requête préparée avec un **INNER JOIN** pour sélectionner les attributs des utilisateurs dont le login est associé à l'identifiant du trajet dans la table **passager**.

```
public static function getAllTrajets() {
    $sql = "SELECT * FROM trajet";
    $rep = Model::$pdo->query($sql);
    $rep->setFetchMode(PDO::FETCH_CLASS, 'Trajet');
    return $rep->fetchAll();
}

public static function findPassagers($id) {
    $sql = "SELECT utilisateur.* FROM utilisateur
        INNER JOIN passager ON utilisateur.login = passager.utilisateur_login
        WHERE passager.trajet_id = :id";

    $req_prep = Model::$pdo->prepare($sql);
    $req_prep->execute(array("id" => $id));

    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'Utilisateur');
    return $req_prep->fetchAll();
}
```

Création d'un nouveau fichier "**testFindUtil.php**" qui sera utilisé pour y faire des tests.

 voiture.php	17/09/2026 18:07	Fichier PHP	3 Ko
 Utilisateur.php	17/09/2026 20:18	Fichier PHP	1 Ko
 Trajet.php	17/09/2026 20:21	Fichier PHP	2 Ko
 testFindUtil.php	17/09/2026 20:24	Fichier PHP	0 Ko

Il permettra instancier la logique de test. Il doit charger les classes, récupérer l'identifiant, exécuter la méthode “**findPassagers(\$id)**”, puis utiliser la méthode “**afficher()**” sur les objets retournés.

```
<?php
require_once 'Model.php';
require_once 'Utilisateur.php';
require_once 'Trajet.php';

$id_test = 3;

$passagers = Trajet::findPassagers($id_test);

echo "<h1>Passagers du trajet n°" . $id_test . "</h1>";

if (empty($passagers)) {
    echo "<p>Aucun passager inscrit pour ce trajet.</p>";
} else {
    foreach ($passagers as $passager) {
        $passager->afficher();
    }
}
?>
```

Résultat :

Passagers du trajet n°3

Passager : Jean Dupont (jdupont)

Création de “**formFindUtil.php**” :

 testFindUtil.php	17/09/2026 20:32	Fichier PHP	1 Ko
 formFindUtil.php	17/09/2026 20:34	Fichier PHP	0 Ko

Il contient un formulaire **HTML** permettant de saisir le numéro d'un trajet, puis d'envoyer cette valeur à la page de traitement “**testFindUtil.php**” via la méthode **GET**.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Recherche de passagers</title>
</head>
<body>
  <form action="testFindUtil.php" method="GET">
    <label for="id_trajet">Identifiant du trajet :</label>
    <input type="text" id="id_trajet" name="id_trajet" required>
    <button type="submit">Rechercher</button>
  </form>
</body>
</html>
```

Modification de “**testFindUtil.php**” pour recevoir le “formulaire” :

```
<?php
require_once 'Model.php';
require_once 'Utilisateur.php';
require_once 'Trajet.php';

if (isset($_GET['id_trajet'])) {
  $id = $_GET['id_trajet'];
  $passagers = Trajet::findPassagers($id);

  echo "<h1>Passagers du trajet n°" . htmlspecialchars($id) . "</h1>";

  if (empty($passagers)) {
    echo "<p>Aucun passager inscrit pour ce trajet.</p>";
  } else {
    foreach ($passagers as $passager) {
      $passager->afficher();
    }
  }
} else {
  echo "<p>Veuillez passer par le formulaire pour chercher un trajet.</p>";
}
?>
```

Résultat :

Identifiant du trajet :

http://localhost/public_html/PHP/TD3/testFindUtil.php?id_trajet=3

Passagers du trajet n°3

Passager : Jean Dupont (jdupont)

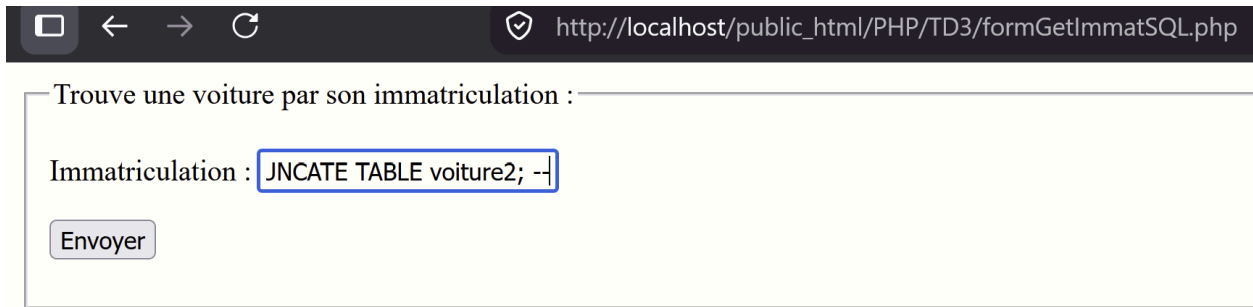
Ensuite, un mini exercice nous propose de faire une injection **SQL**.

On va créer une table temporaire (fournie avec le **TD**) qui s'appelle **"voiture2.sql"** et un fichier **PHP "formGetImmatSQL.php"** :

The screenshot shows the phpMyAdmin interface. On the left, the database structure is visible, with 'voiture2' highlighted under the 'mvc_php_td2' database. The main panel displays the SQL query `SELECT * FROM `voiture2`` and its results. The results are shown in a table with columns: `immatriculation`, `marque`, and `couleur`. There are 5 rows of data. Each row has checkboxes for 'Éditer', 'Copier', and 'Supprimer'.

	immatriculation	marque	couleur
☐ Éditer ☐ Copier ☐ Supprimer	12345678	Ferrari	vert
☐ Éditer ☐ Copier ☐ Supprimer	259641	Renault	Noire
☐ Éditer ☐ Copier ☐ Supprimer	710097	Tesla	Bleu nuit
☐ Éditer ☐ Copier ☐ Supprimer	8369210	Tesla	Bleu nuit
☐ Éditer ☐ Copier ☐ Supprimer	abc124az	Renault	bleu

La page ressemble à ceci, elle n'est pas protégée contre les injections **SQL**, on va y injecter **; TRUNCATE TABLE voiture2; --**.

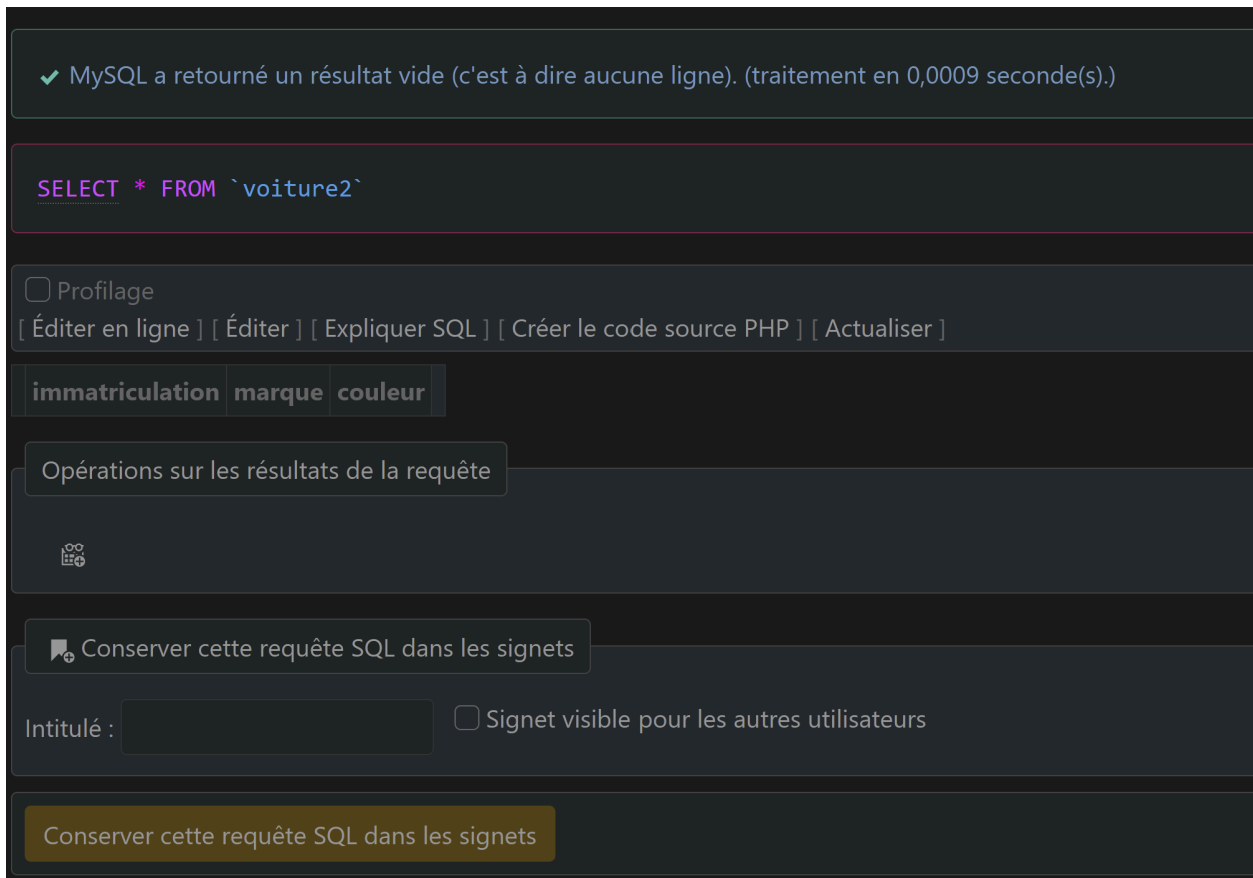


Trouve une voiture par son immatriculation :

Immatriculation :

J'effectue la requête "SELECT * from voiture2 WHERE immatriculation=''; TRUNCATE TABLE voiture2; --"

La table a été vidée :



✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0009 seconde(s))

```
SELECT * FROM `voiture2`
```

☐ Profilage

[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

immatriculation marque couleur

Opérations sur les résultats de la requête



 Conserver cette requête SQL dans les signets

Intitulé : ☐ Signet visible pour les autres utilisateurs

Conserver cette requête SQL dans les signets

Exercice 8

Il faut récupérer tous les trajets auxquels un utilisateur spécifique est inscrit en tant que passager, en utilisant une jointure **SQL**.

Ajout d'une fonction "**findTrajets()**" dans "**Utilisateur.php**" :

```
public static function findTrajets($login) {
    $sql = "SELECT trajet.* FROM trajet
           INNER JOIN passager ON trajet.id = passager.trajet_id
           WHERE passager.utilisateur_login = :login";

    $req_prep = Model::$pdo->prepare($sql);
    $req_prep->execute(array("login" => $login));

    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'Trajet');
    return $req_prep->fetchAll();
}
```

Création de "**testFindTraj.php**" et "**formfindTraj.php**" :

 testFindTraj.php	17/09/2026 21:00	Fichier PHP	0 Ko
 formFindTraj.php	17/09/2026 21:01	Fichier PHP	0 Ko

Contenu de "**formfindTraj.php**" :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Recherche de trajets</title>
</head>
<body>
    <form action="testFindTraj.php" method="GET">
        <label for="login">Login de l'utilisateur :</label>
        <input type="text" id="login" name="login" required>
        <button type="submit">Chercher ses trajets</button>
    </form>
</body>
</html>
```

Ajout de la fonction “**afficher()**” dans “**Trajet.php**” :

```
public function afficher() {  
    echo "<p>Trajet n°" . htmlspecialchars($this->id) . " : de " . htmlspecialchars($this->depart) . " à " . htmlspecialchars($this->arrivee) . "</p>";  
}
```

Contenu de “**testFindTraj.php**” :

```
<?php  
require_once 'Model.php';  
require_once 'Utilisateur.php';  
require_once 'Trajet.php';  
  
if (isset($_GET['login'])) {  
    $login = $_GET['login'];  
    $trajets = Utilisateur::findTrajets($login);  
  
    echo "<h1>Trajets du passager : " . htmlspecialchars($login) . "</h1>";  
  
    if (empty($trajets)) {  
        echo "<p>Cet utilisateur n'est inscrit à aucun trajet.</p>";  
    } else {  
        foreach ($trajets as $trajet) {  
            $trajet->afficher();  
        }  
    }  
} else {  
    echo "<p>Veuillez passer par le formulaire pour chercher un utilisateur.</p>";  
}  
?>
```

Résultat :



The screenshot shows a web browser window with the address bar displaying "http://localhost/public_html/PHP/TD3/formFindTraj.php". Below the address bar, there is a search form with the label "Login de l'utilisateur :". The input field contains the text "jdupont". To the right of the input field is a button labeled "Chercher ses trajets". Below the search form, the page displays the results in a large, bold, black serif font. The first line of the results is "Trajets du passager : jdupont". The second line is "Trajet n°3 : de Briançon à Gap".

Exercice 9

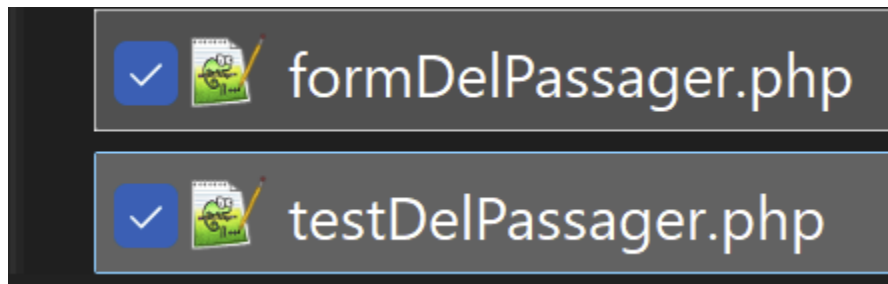
On souhaite désinscrire un utilisateur d'un trajet et inversement.

Ajout de **"deletePassager()"** dans **"Trajet.php"** :

```
public static function deletePassager($data) {
    $sql = "DELETE FROM passager WHERE trajet_id = :trajet_id AND utilisateur_login = :utilisateur_login";
    $req_prep = Model::$pdo->prepare($sql);

    $req_prep->execute(array(
        "trajet_id" => $data['trajet_id'],
        "utilisateur_login" => $data['utilisateur_login']
    ));
}
```

Création de **"formDelPassager.php"** et **"testDelPassager.php"** :



Contenu de **"formDelPassager.php"** :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Désinscription d'un trajet</title>
</head>
<body>
    <form action="testDelPassager.php" method="POST">
        <label for="trajet_id">ID du trajet :</label>
        <input type="text" id="trajet_id" name="trajet_id" required><br><br>

        <label for="utilisateur_login">Login de l'utilisateur :</label>
        <input type="text" id="utilisateur_login" name="utilisateur_login" required><br><br>

        <button type="submit">Désinscrire l'utilisateur</button>
    </form>
</body>
</html>
```

Contenu de “testDelPassager.php” :

```
<?php
C:\xampp\htdocs\public_html\PHP\TD3\Trajet.php
require_once 'Trajet.php';

if (isset($_POST['trajet_id']) && isset($_POST['utilisateur_login'])) {
    $data = array(
        'trajet_id' => $_POST['trajet_id'],
        'utilisateur_login' => $_POST['utilisateur_login']
    );
    Trajet::deletePassager($data);

    echo "<p>L'utilisateur " . htmlspecialchars($data['utilisateur_login']) . " a été désinscrit du trajet n°" . htmlspecialchars($data['trajet_id']) . " avec succès.</p>";
} else {
    echo "<p>Erreur : Veuillez renseigner le formulaire de désinscription.</p>";
}
?>
```

Résultat :

Table passager actuelle :

				trajet_id	utilisateur_login
☐	Éditer	Copier	Supprimer	3	jdupont

http://localhost/public_html/PHP/TD3/formDelPassager.php

Afficher le panneau latéral (Alt+Ctrl+Z)

ID du trajet :

Login de l'utilisateur :

L'utilisateur jdupont a été désinscrit du trajet n°3 avec succès.

La table a été vidée (il n'y avait qu'une seule colonne).

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0008 seconde(s).)

```
SELECT * FROM `passager`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

trajet_id	utilisateur_login
-----------	-------------------

Opérations sur les résultats de la requête

Conserver cette requête SQL dans les signets

Intitulé : ☐ Signet visible pour les autres utilisateurs

Conserver cette requête SQL dans les signets

Conclusion

Ce TD nous a appris à utiliser les jointure via l'interface **phpMyAdmin** ainsi que de les utiliser en **PHP**.